

Defense-in-Depth Runtime Safety in Move

Victor Gao, Wolfgang Grieskamp, Vineeth Kashyap, George Mitenkov,
Teng Zhang, Runtian Zhou, Andrea Cappa, and Marco Illadi

Aptos Labs, Palo Alto, USA

Abstract. Move is a smart-contract language used to execute transactions on the Aptos blockchain. Move programs execute in a sandboxed VM as typed bytecode. The VM statically verifies a few foundational safety properties — in particular type safety and reference safety — ahead of execution, during code loading time. In principle, this design gives strong guarantees for Move. However, the static verification logic is complex and continually evolving with changes in the language. Just like any other software, the static verifier is not immune to bugs: historically, there have been subtle errors in the verifier that can let rule-violating bytecode through. In a live blockchain setting, such bugs are not academic: a missed type or reference rule violation can translate directly into loss of assets, forged authority, or unrecoverable corruption of on-chain state. For this reason, Aptos relies on defense in depth, in the form of *runtime safety checks* that independently verify the critical invariants during execution, providing protection against both compromised bytecode and latent verifier bugs. This paper describes the runtime safety checks in place for Move on Aptos.

Keywords: Move · Smart contracts · Runtime verification · Type safety · Reference safety.

1 Introduction

The Move on Aptos programming language was originally developed for the Libra/Diem blockchain at Meta [1, 4, 5] and is now deployed at scale on Aptos [3], a high-throughput Layer-1 blockchain that settles transactions for billions of dollars in user assets on chain. A defining trait of the language is its resource-oriented type discipline: digital assets are represented as Move *resources* whose ownership and movement are tracked by linear-style abilities (`copy`, `drop`, `store`, `key`), ruling out accidental duplication or destruction at compile time. The Aptos platform builds on this foundation with a system of objects, parallel execution, and a rich on-chain framework written in Move itself [2, 16].

Beyond its type discipline, Move has been designed for *offline formal verification*: the Move Prover translates Move programs and their specifications, written in the Move Specification Language, into verification conditions discharged by SMT solvers [7, 9]. The Move Prover can verify rich properties about the smart contracts themselves, but it is inherently offline, applied prior to deployment of the contracts.

Move source programs are compiled into typed bytecode executed by the Move VM. The Move VM performs *static bytecode verification* every time a module is loaded, enforcing well-formedness, typing, and Move’s reference discipline — a Rust-like [12, 14] aliasing regime detailed in §2. The reference safety verifier [6] is particularly intricate: it maintains a borrow graph that conservatively approximates the aliasing relationships entailed by control- and data-flow using abstract interpretation. Implementing and maintaining the correctness of various static bytecode verification passes is a delicate engineering task, especially with the rapid evolution of the Move language. Since the public launch of Aptos, Move has been extended to include enums, closures, and public structs, among other features. These language features interact non-trivially with the static verifier. Given the high stakes, Aptos runs a bug bounty program, which has surfaced verifier bugs in the past.

In a blockchain setting, executed transactions cannot be reversed, hand-crafted compiler-bypassed bytecode can be deployed by adversaries. Static verification is thus a single point of failure: one exploited bug can have irreversible and catastrophic consequences. Therefore, as a practical security measure, the Move VM implements defense-in-depth *runtime safety checks* that independently verify the critical invariants during execution, after the bytecode has passed static verification.

In this paper, motivated by real-world examples, we describe three groups of runtime checks. *Type checks* (§3.1) verify that the dynamically tracked type of every value matches the static type the bytecode expects to consume. *Ability checks* (§3.2) verify that operations such as copy, drop, or storage in global state are only performed on values whose declared abilities permit them. *Reference checks* (§3.3) verify the same freedom from dangling references and aliased mutable access that the static verifier is supposed to enforce.

We then describe two techniques employed to reduce the runtime costs of these checks: the VM supports *trusted code* (§4) that waives them under certain conditions, and an *asynchronous* checking mode (§5) leverages idle workers in Block-STM [8] (Aptos’s speculative parallel executor). We present an evaluation of the performance impact of the checks on real-world benchmarks in §6.

2 Move on Aptos

We briefly summarize key aspects of Move on Aptos [3, 5].

- *Typing and generics.* Move is strongly statically typed. Generic functions and types are instantiated by substitution at the call site. Substitution is carried through transitively: accesses to type-indexed global storage (below) are keyed by concrete types at runtime.
- *Abilities.* Each type carries a subset of the four abilities **copy**, **drop**, **store**, and **key**, which gate the corresponding operations. Types without **drop** or **copy** are linear: every value of such a type must be moved, globally stored, or explicitly destructured.

- *Algebraic data types.* Structs are nominal product types; enums are nominal sums whose variants carry payloads. Pattern matching may bind references directly into a variant’s fields.
- *Modular encapsulation.* Privileged operations on a module-private type — construction, destruction, and field access — are confined to its defining module, so module-external code can manipulate values only through module provided functions.
- *References.* Immutable ($\&T$) and mutable ($\&\text{mut } T$) references obey a static aliasing discipline comparable to Rust’s [12, 14]: two live mutable references cannot point to overlapping memory, and references cannot outlive the values they borrow. In contrast to Rust, Move (a) does allow mutable references to global storage, but (b) has no reference lifetimes and does not allow references to be stored in structs or enums or captured by closures.
- *Global storage.* Global state is a map indexed by (resource type, address) pairs. Resources of types with the `key` ability are accessed through $T[\text{addr}]$, $\&T[\text{addr}]$, and $\&\text{mut } T[\text{addr}]$, with the same linear-ownership discipline as for local values.

3 Runtime Safety Checks

In this section, we give a conceptual overview of the runtime checks, motivated by real examples of severe security bugs that they are designed to prevent.

3.1 Type Safety

Move has nominal typing: two structs with identical layout but different names are distinct types. A *type-confusion* exploit breaks that distinction, and can be very damaging when one of the two types is privileged. Consider the example:

```

module 0x1::admin {
    struct Capability(address);
    public fun admin_action(c: &Capability) {
        /* Capability gated action */
    }
}

module 0xdeadbeef::attacker {
    struct FakedCapability(address);
}

```

The attacker can freely construct a `FakedCapability` (they own the type); a type confusion exploit would allow passing it to `admin_action` as if it were a `Capability`.

Example When vector operations were added to the VM during Move’s original development at Meta, the bytecode verifier was not extended with the matching check on the vector element type for `push`. The compiler did perform this check, so no input source program could reach the problem. However, an attacker could bypass the compiler and directly construct bytecode, conceptually doing the following:

```

let v: vector<Capability> = vector::empty();
v.push_back(FakedCapability(victim)); // should be rejected
admin_action(&v[0]); // runs on a faked type

```

The two types share a layout, so program runs without corruption, but the gated action runs on a faked type. This bug was found as part of the Aptos bug bounty program before going into production.

Performing the Checks To prevent type confusion, in addition to the byte-code verifier, we perform type checks at runtime. A type stack is maintained in parallel to the operand stack of the Move VM. Because the Move VM’s registers are typed, the type of every value is known: whenever a value is pushed on the operand stack, its type is pushed on the type stack. Whenever a value is consumed, by being stored into a register or used by an operation, the corresponding type is popped and compared with the expected type of the register or operation; a mismatch is type confusion and raises a runtime error.

3.2 Ability Safety

Bugs not enforcing copy ability are very damaging. A value whose type lacks copy is meant to be unique: this is what makes the `Capability` of §3.1 a single-issue permission, and what makes a Coin balance impossible to counterfeit. A runtime that produces a fresh value of a no-copy type silently duplicates that resource.

Example Move’s ability system permits an assignment to downgrade a value’s abilities (i.e., width subtyping over the ability set, which is sound). But due to the classic invariance of mutable references [17], applying this rule through a mutable reference is unsound. When Move was extended with closures, there was a corner case (see example below) which did not enforce invariance. An exploit could capture a no-copy resource `r` inside a no-copy closure `h`, store that closure into a variable `f` of copy-able closure type, and then copy `f` freely, calling it multiple times to duplicate the no-copy resource `r`.

```

struct NoCopy(u64) has drop;
public fun assign<T: drop>(ref: &mut T, x: T) {
    *ref = x; // drops the old value
}
// — entry point for the attack —
let r = NoCopy(42);
let h: || has drop = || {
    // capture and use ‘r’, valid in a no-copy closure
};
let f: || has drop + copy = || {}; // no-op closure
assign<|| has drop>(&mut f, h); // bug: not invariant
let g = copy f; f(); g();

```

This bug was found as part of pre-production security auditing.

Performing the Checks The VM performs runtime checks for violations of each of the abilities `copy`, `drop`, `key`, and `store`. Most of those checks are relatively straightforward and can be directly performed when the corresponding

instruction is executed. The exception is `drop`, which requires the VM to track which values have not been consumed on function exit. The runtime checks for `drop` are more precise (thus would allow more programs to pass) than the static checks, as the latter has to conservatively reject if a no-`drop` local is maybe not consumed on some control flow path (even if that path is infeasible), while the former only checks the actually executed path.

3.3 Reference Safety

The mutable-aliasing rule is critical in the presence of `enum` types: a reference into a variant payload is type-safe only while the enum still holds that variant. If two mutable references to the same enum can simultaneously be live, one can switch the variant under the other.

Example Consider the following code:

```
enum E { V1(Capability), V2(FakedCapability) }

fun aliased_swap(a: &mut E, b: &mut E, faked: &mut E) {
    // faked is V2(FakedCapability(victim))
    match (a) {
        E::V1(c) => {
            mem::swap(b, faked); // *a is now V2(..)
            admin_action(c); // c still &Capability
        },
        _ => {},
    }
}
```

A call of the form `aliased_swap(p, q, ...)` where `p` and `q` alias should normally be rejected by the bytecode verifier. However, there was a bug in the bytecode verifier’s borrow analysis which did not correctly identify borrow graph edges `e.f1` and `e.f2` from different variants refer to the same physical field at the same offset, in hand-crafted malicious bytecode. This allowed creating mutable references to the same field offset in the enum, eventually enabling a call to `aliased_swap` with aliased mutable references. This bug was discovered via the bug bounty program after the introduction of enum types.

Performing the Checks Checking reference safety at runtime for Move-style borrow semantics is a novel problem (see §7). The challenge is that during construction of references, mutable reference aliasing *is* allowed. Only once those references are used must aliasing be forbidden. As an example, consider the following valid code:

```
struct S {a: u64, b: u64}
let s: S;
let r1 = &mut s.a;
let r2 = &mut s.b;
*r1 = 1; *r2 = 2; // r1, r2 reference disjoint fields
```

The corresponding stack-machine bytecode fragment is shown in Fig. 1.

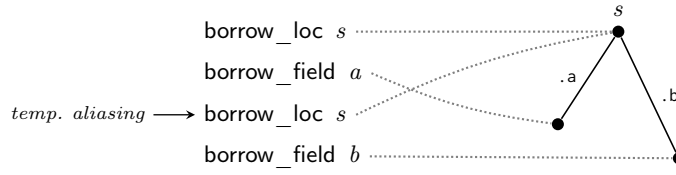


Fig. 1: Each reference-producing operation in the bytecode (left) is linked by a dotted line to the access-path-tree node (right) whose place the reference it pushes denotes. Both `borrow_loc s` operations denote the root *s* (the temporary aliasing) while the two `borrow_field` operations denote the disjoint child places reached by the field-offset edges *.a* and *.b*.

Thus while we have a mutable reference to *s.a* on the stack, we temporarily create a new mutable reference to the whole of *s*, overlapping with the existing one (Fig. 1). Only once this reference is consumed to create the reference to *s.b* is the aliasing eliminated. This property makes a naive approach to runtime reference safety checking, for example via reference counting, infeasibly strict.

Our checker instead executes an *abstract shadow semantics* in lockstep with execution. The VM’s operand stack, locals, and call frames are mirrored by a shadow state in which every non-reference value is opaque and every reference is a fresh identifier ρ minted at its borrow site. The shadow state never inspects runtime values or memory addresses; conceptually, it is an abstract interpretation of the one path the program actually takes, embedded along with the concrete execution. The following ideas underpin the scheme in the shadow execution.

State representation. References point into trees of *places*, not at each other. Each function lazily grows *access path trees* rooted at its locals, at the global resource types it borrows, and at the values behind its reference parameters. Edges are physical field offsets: fields of different enum variants that share an offset share an edge. A reference ρ is a pointer to a node (representing a place) in one of these trees, tagged with its mutability. Every aliasing question reduces to an ancestor/descendant relation between the places two references denote; no borrow genealogy among the references themselves is recorded.

Poison eagerly, fail lazily. A destructive operation, like a write through a mutable reference or the implicit invalidation of a local by `move_loc`, poisons every other reference whose places it invalidates. No error is raised at the operation itself. If a poisoned reference is later *used* — read, written, borrowed through, or passed to a callee — a transaction-aborting error is raised. Poisoned references may still be moved around or dropped without error. Overlapping references that never destructively interact are never flagged at all, admitting the temporary aliasing illustrated above.

Function modular. The checker is modular and compositional across function boundaries. A call consumes its reference arguments, checks at the call site that the places reachable from each mutable argument are disjoint from those reach-

able from every other reference argument, and hands the callee fresh trees rooted at its own reference parameters. Passing a mutable reference into a call counts as a destructive operation, since the callee may mutate arbitrarily through it. Upon return, every returned reference must denote a place inside the tree of some reference parameter; its access path is replayed into the corresponding tree of the caller. Returned mutable references must also denote mutually disjoint places. The checker thereby learns precisely which parameter each returned reference derives from, along the executed path, whereas the static analysis must conservatively assume that a returned reference may borrow from every reference argument.

Taken together, these ideas yield a *relaxed dynamic semantics* of the static borrow analysis: bytecode accepted by a correct verifier is expected to pass the runtime checks on every execution, while the checker, observing only the executed path, does not have the join and call-summary over-approximations of the static analysis and thus admits strictly more behaviors.

Revisiting the example, the bytecode fragment above runs successfully: the second `borrow_loc s` briefly creates a second mutable reference at the root of the tree for `s`, but that reference is consumed by `borrow_field b` before any destructive write fires, so nothing is poisoned. The exploit that mints the overlapping references is caught by poisoning instead: tree edges are physical field offsets, so the borrow paths `e.f1` and `e.f2` land on the same node by construction. The destructive `mem::swap` then poisons the still-live payload reference `c` sitting at a descendant place, and the `admin_action(c)` aborts as it is considered `c`'s use.

4 Trusted Code

Runtime safety checks impose a performance overhead. The Move VM supports the concept of *trusted code* which waives those checks. Currently, trusted code deployment underlies Aptos Foundation governance, and is only allowed for the Aptos framework, whose deployment needs to be approved by the Aptos operators. We are also investigating staking-based models for trusted code.

The main property of trusted code is that the governance model gives higher confidence in the integrity of the code. This does not exclude bugs in the VM, but it does reduce the likelihood of maliciously engineered exploits.

When executing trusted code, the checks discussed in the previous section are disabled. Since checks like type safety collect information during execution, care must be taken to correctly model the transitions from untrusted into trusted code and vice versa. For example, trusted code does not use the type stack, but untrusted code expects the type of a return value from a function call on the type stack, so this information needs to be synthesized when returning from a call to a trusted function into untrusted code.

5 Asynchronous Checking

Aptos executes blocks of transactions in parallel using *Block-STM* [8], an optimistic concurrency-control protocol in which a pool of worker threads speculatively runs transactions out of order and re-validates them against later-published writes; incarnations that lose a race are aborted and replayed. In

Table 1: Throughput (txns/sec, higher is better) under type and ability checking. Slowdown vs the no-checks baseline presented in parentheses, “waiver” skips trusted framework code (§4); “asynchronous” defers checks to trace replay (§5).

workers	baseline	synchronous		asynchronous	
		no-waiver	waiver	no-waiver	waiver
2	864	611 (−29.3%)	739 (−14.5%)	513 (−40.6%)	624 (−27.7%)
4	1427	1026 (−28.1%)	1236 (−13.4%)	914 (−35.9%)	1088 (−23.7%)
8	1915	1350 (−29.5%)	1620 (−15.4%)	1427 (−25.5%)	1598 (−16.5%)
16	2019	1427 (−29.3%)	1691 (−16.2%)	1690 (−16.3%)	1799 (−10.9%)
32	1904	1363 (−28.4%)	1621 (−14.9%)	1633 (−14.2%)	1716 (−9.9%)

Table 2: Throughput (txns/sec) under dynamic reference safety checking: no waiver, synchronous, in isolation (type and ability checks disabled).

workers	checks off	checks on	slowdown
2	858	630	−26.5%
4	1430	1054	−26.3%
8	1918	1370	−28.6%
16	2011	1450	−27.9%
32	1893	1375	−27.4%

practice most blocks do not saturate the worker pool, and aborted-and-restarted incarnations leave threads idle for non-trivial windows, so a substantial slice of worker capacity remains unused during normal operation.

We use that idle capacity to run the runtime safety checks *asynchronously*. During execution the interpreter trails a compact *execution trace* for each transaction, containing no values, only control-flow shape.

When a transaction’s commit-time incarnation finalizes, an idle worker replays its trace by abstractly interpreting the bytecode along the recorded path — following the recorded branches, dispatching the recorded closures — and runs the type and ability checks of §3.1 and §3.2 on the fully-substituted types. A violation found during replay aborts the block before its writes become observable to subsequent blocks. Because asynchronous checks are run on successful transactions only, speculative executions and aborted transactions do not pay the cost of these checks. Because the checks are almost always expected to pass on the static verifier admitted bytecode, except in the low probability event of a latent bug identified in the verifier being exploited, computation done while executing a transaction is not wasted because of failed checks.

6 Performance

Our real-world benchmark is a perpetual-futures DEX with 100 markets, driven by a weighted mix of oracle price updates, bulk market-maker orders, and retail taker orders, executed by Block-STM [8] on a 60-vCPU Google Cloud instance with 235 GB of RAM as the worker pool varies from 2 to 32 threads. Each reported number is the mean of three runs; run-to-run spread is below 1.3%.

Table 1 reports the cost of type and ability checking, which closely share their implementation. Synchronously, the combined checks cost 28–29% of throughput, uniformly across worker counts. The trusted-code waiver of §4 roughly halves this to 13–16%, as about two thirds of executed instructions on this workload fall into trusted framework code. The asynchronous mode of §5 trades a small tracing overhead for deferred checking on idle workers: it is slower than synchronous below 8 workers, reaches parity at 8, and at 16–32 brings the cost to 10–11%, the preferred configuration at production concurrency.

Table 2 reports reference safety checking in isolation: 26–29%, again uniform across worker counts. While the reference checker is completely functional, it is not yet fully optimized and does not yet support the trusted-code waiver or asynchronous deferral, and thus not yet live in production. The type-check numbers above set a reasonable expectation for what optimizing and extending these mechanisms should recover.

7 Related Work and Conclusion

Runtime bytecode verification at module load time is standard for typed managed virtual machines: the Java Virtual Machine [13] and the Common Language Runtime [15] both run a verifier at class-load time to check well-formedness, typing, and stack discipline before bytecode is executed, and the sibling Move VMs deployed elsewhere — notably the original Diem MoveVM [4] and Sui’s variant — inherit Move’s bytecode verifier. None of these platforms maintains a *second* layer of runtime safety checks during execution: once the verifier admits a module, its dynamic semantics is trusted. At the opposite end of the spectrum, dynamically typed languages such as JavaScript or Python perform the runtime type checks of §3.1, but lack the *first* static layer. The Ethereum Virtual Machine [10, 19] and Solana’s SVM sit in a third regime: their bytecode is untyped (a stack of 256-bit words for the EVM; raw memory regions for the SVM), so neither a type-based static verifier nor a runtime type discipline applies. Aptos’s combination of a static bytecode verifier with a complementary runtime layer is, to our knowledge, unique among production VMs and goes beyond the safety guarantees of any other smart-contract platform.

The dynamic reference safety checks (DRSC) we describe are novel. The closest related works on Tree Borrows and Stacked Borrows [11, 18] (TB/SB) propose to clarify Rust’s unsafe aliasing rules. DRSC is designed not to have false positives over the static verifier, whereas TB/SB do not share that goal. TB/SB can also tolerate two orders or more of magnitude in performance overhead: they are not expected to be run in production or huge fuzzing workloads. Unlike DRSC, they maintain provenance genealogies among references and are not functional-modular. Also see the difference in Move and Rust’s references in §2.

The dynamic semantics is also interesting as a reference model for what reference safety means, as most static analyses are over-approximations determined by the particular method chosen, and a canonical semantics does not exist. More work is needed to fully formalize and categorize dynamic reference safety.

In addition to providing a second layer of defense against latent bugs in the static verifier, the runtime checks also have a valuable role in fuzzing the VM: they provide a semantic and precise oracle for finding bugs in the static verifier.

Disclosure of Interests. All authors are employees of Aptos Labs and may hold equity in the company. The work described is implemented in the Aptos blockchain.

References

1. Amsden, Z., Arora, R., Bano, S., Baudet, M., Blackshear, S., Bothra, A., Cabrera, G., Catalini, C., Chalkias, K., Cheng, E., Ching, A., Chursin, A., Danezis, G., Giacomo, G.D., Dill, D.L., Ding, H., Doudchenko, N., Gao, V., Gao, Z., Garillot, F., Gorven, M., Hayes, P., Hou, J.M., Hu, Y., Hurley, K., Lewi, K., Li, C., Li, Z., Malkhi, D., Margulis, S., Maurer, B., Mohassel, P., de Naurois, L., Nikolaenko, V., Nowacki, T., Orlov, O., Perelman, D., Pott, A., Proctor, B., Qadeer, S., Rain, Russi, D., Schwab, B., Sezer, S., Sonnino, A., Venter, H., Wei, L., Wernerfelt, N., Williams, B., Wu, Q., Yan, X., Zakian, T., Zhou, R.: The Libra Blockchain. <https://developers.libra.org/docs/the-libra-blockchain-paper> (2019)
2. Aptos Labs: The Aptos Framework Book. <https://aptos-labs.github.io/framework-book/> (2023), accessed: 2026-05-09
3. Aptos Labs: The Move on Aptos Book. <https://aptos-labs.github.io/move-book/> (2023), accessed: 2026-05-09
4. Blackshear, S., Cheng, E., Dill, D.L., Gao, V., Maurer, B., Nowacki, T., Pott, A., Qadeer, S., Rain, Russi, D., Sezer, S., Zakian, T., Zho, R.: Move: A Language With Programmable Resources (2019), <https://developers.libra.org/docs/move-paper>
5. Blackshear, S., Dill, D.L., Qadeer, S., Barrett, C.W., Mitchell, J.C., Padon, O., Zohar, Y.: Resources: A safe language abstraction for money (2020), <https://arxiv.org/abs/2004.05106>
6. Blackshear, S., Mitchell, J.C., Nowacki, T., Qadeer, S.: The Move borrow checker. CoRR **abs/2205.05181** (2022), <https://arxiv.org/abs/2205.05181>
7. Dill, D.L., Grieskamp, W., Park, J., Qadeer, S., Xu, M., Zhong, J.E.: Fast and reliable formal verification of smart contracts with the move prover. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2022). Lecture Notes in Computer Science, vol. 13243, pp. 183–200. Springer (2022). https://doi.org/10.1007/978-3-030-99524-9_10
8. Gelashvili, R., Spiegelman, A., Xiang, Z., Danezis, G., Li, Z., Malkhi, D., Xia, Y., Zhou, R.: Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing. In: Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP 2023). pp. 232–244. ACM (2023). <https://doi.org/10.1145/3572848.3577524>
9. Grieskamp, W., Zhang, T., Kashyap, V., Silverman, J.: Formal verification of imperative first-class functions in Move. CoRR **abs/2605.10007** (2026), <https://arxiv.org/abs/2605.10007>
10. Hildenbrandt, E., Saxena, M., Rodrigues, N., Zhu, X., Daian, P., Guth, D., Moore, B.M., Park, D., Zhang, Y., Stefanescu, A., Rosu, G.: KEVM: A complete formal semantics of the ethereum virtual machine. In: CSF. pp. 204–217. IEEE Computer Society (2018)
11. Jung, R., Dang, H.H., Kang, J., Dreyer, D.: Stacked borrows: An aliasing model for Rust. Proc. ACM Program. Lang. **4**(POPL), 41:1–41:32 (2020)

12. Jung, R., Jourdan, J.H., Krebbers, R., Dreyer, D.: RustBelt: Securing the foundations of the Rust programming language. *Proc. ACM Program. Lang.* **2**(POPL), 66:1–66:34 (2018)
13. Lindholm, T., Yellin, F.: *The Java Virtual Machine Specification*. Addison-Wesley (1997)
14. Matsakis, N.D., Klock, II, F.S.: The Rust Language, nouri = <http://doi.acm.org/10.1145/2692956.2663188>. *Ada Lett.* **34**(3), 103–104 (Oct 2014). <https://doi.org/10.1145/2692956.2663188>
15. Meijer, E., Wa, R., Gough, J.: *Technical overview of the common language runtime* (2000)
16. Park, J., Zhang, T., Grieskamp, W., Xu, M., Giacomo, G.D., Chen, K., Lu, Y., Chen, R.: Securing Aptos Framework with Formal Verification. In: 5th International Workshop on Formal Methods for Blockchains (FMBC 2024). Open Access Series in Informatics (OASICS), vol. 118, pp. 9:1–9:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2024). <https://doi.org/10.4230/OASICS.FMBC.2024.9>
17. Pierce, B.C.: *Types and Programming Languages*. MIT Press, Cambridge, MA, USA (2002)
18. Villani, N., Dang, H.H., Jourdan, J.H., Jung, R., Dreyer, D.: Tree borrows. *Proc. ACM Program. Lang.* **9**(PLDI) (2025)
19. Wood, G.: *Ethereum: A secure decentralised generalised transaction ledger* (2014), <https://ethereum.github.io/yellowpaper/paper.pdf>